Task 4: Convolutional Neural Networks

Building a CNN in PyTorch, and comparing it to Task 3

---

1. What you are building

A CNN that classifies **CIFAR-10** — 60,000 colour images, 32×32 pixels, 10 classes (plane, car, bird, cat, deer, dog, frog, horse, ship, truck).

This task is **PyTorch only**. You already wrote forward and backward by hand in Task 3, so there is nothing to prove by doing it again. What matters here is the architecture and what it buys you.

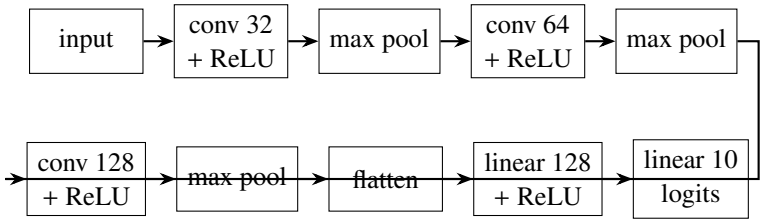
You will build two models and compare them:

1. A **CNN** — the architecture specified below.
2. An **MLP** — fully connected, no convolutions, on the same data.

Why CIFAR-10 and not MNIST

MNIST is too easy. A plain MLP gets 98% on it, so a CNN has nothing to show you. CIFAR-10 has colour, texture, backgrounds and objects in different positions — and an MLP struggles badly with it. That gap between the two models *is* the lesson of this task.
If both models land in the same place, something is wrong.

2. The CNN architecture



- Input: 3 channels, 32×32.
- Three convolutional blocks, each one: convolution → ReLU → max pool.
- Output channels of the three convolutions: **32**, then **64**, then **128**.
- Every convolution: kernel 3×3, stride 1, padding 1.
- Every pooling layer: max pool 2×2, stride 2.

- Then flatten, a fully connected layer with **128** units and ReLU, and a final fully connected layer with **10** outputs.
- The final layer produces raw logits. Nothing after it.

3. Before you write any code

Work out the shape after every layer, **on paper**, using

$$\text{out} = \left\lfloor \frac{\text{in} + 2p - k}{s} \right\rfloor + 1$$

Layer	Output shape	# parameters
input	$3 \times 32 \times 32$	0
conv1 (→32)	32 * 32 * 32	896
pool1	16 * 16 * 32	0
conv2 (→64)	16 * 16 * 64	18496
pool2	8 * 8 * 64	0
conv3 (→128)	8 * 8 * 128	73856
pool3	4 * 4 * 128	0
flatten	2048	0
linear1	2048 * 128	262272
linear2	128 * 10	1290

You will need the flatten size to write the first fully connected layer at all, so this table is not optional homework — it is the thing that lets you write the code.

Check it afterwards against the model summary and a parameter count. If the two disagree, your table is what needs fixing.

Counting the parameters of a convolution

A single filter has $C_{\text{in}} \times k \times k$ weights plus one bias. A layer with C_{out} output channels holds C_{out} such filters.
Notice what the *spatial* size of the input (32×32, 16×16, . . .) contributes to that count: nothing. The same filter is applied at every position, so the number of parameters is independent of how big the image is. That is **parameter sharing**, and it is half the reason this network works.

Why padding 1 with a 3×3 kernel

Substitute $k = 3$, $p = 1$, $s = 1$ into the formula above and see what you get. The convolution does not shrink the image at all — this is called "same" padding.
That means the only thing changing the spatial size in this network is the pooling, which halves it each time. Separating the two jobs like this — convolution learns features, pooling reduces size — makes an architecture much easier to reason about than letting both shrink the image at once.

The depth of a filter

The first convolution has 3 input channels, so each of its filters is $3 \times 3 \times 3$ — it spans all three colour channels at once, not one at a time. Each filter slides over the whole image and produces one output channel, called a *feature map*. 32 filters, 32 feature maps.
The second convolution receives 32 channels, so each of *its* filters is $3 \times 3 \times 32$. The depth of a filter is always the number of channels coming in. Get this straight before coding and most shape errors disappear.

What flatten is for

A convolution works on a 4-D tensor: batch, channels, height, width. A fully connected layer wants 2-D: batch, features. Flattening collapses everything after the batch dimension into a single vector.
The size of that vector is the last three numbers of your table multiplied together — and you need it to define the first fully connected layer. Work it out rather than guessing; if you get it wrong the code will crash and the error message will tell you the shape it actually received.

4. The MLP to compare against

Same data, same training settings, no convolutions:

- Flatten the image straight away — $3 \times 32 \times 32 = 3072$ inputs.
- Three fully connected layers: $3072 \to 512 \to 256 \to 10$, with ReLU between them.
- Raw logits at the end, same as the CNN.

Count its parameters too.

5. Training setup — use these exactly

- Dataset: CIFAR-10 from `torchvision`, train and test splits.
- Preprocessing: convert to tensor, then normalise each channel with mean 0.5 and standard deviation 0.5.
- Batch size: 64
- Optimizer: Adam, learning rate 10^{-3}
- Loss: cross-entropy
- Epochs: 10
- Seed: 0

The training loop is the same one you wrote in Task 3 — zero the gradients, forward, loss, backward, step. Reuse it.

Two reminders from Task 3

No softmax at the end of `forward`. PyTorch's cross-entropy loss takes raw logits and applies the softmax internally.
Use a GPU if you have one, and remember to move both the model and every batch onto it. On CPU this still runs, just slowly — roughly 1–2 minutes per epoch.

6. The comparison

Report both models side by side:

	MLP	**CNN**
Total trainable parameters	1,707,274	356,714
Test accuracy	54 %	75 %
Time per epoch	35.94 seconds	76.61 seconds

The result you should find surprising

The MLP has several times *more* parameters than the CNN — and it performs clearly worse. This contradicts Lecture 1, where more capacity meant more expressive power. So the question is not which model is bigger. It is: what does the CNN **assume** about images that the MLP does not, and why is that assumption worth more than millions of extra parameters? To get at it, think about what happens to each model when you shift an image two pixels to the right.

7. One thing to look at

Take your trained CNN, pull out the weights of the **first** convolutional layer, and display its 32 filters as small 3×3 colour patches (normalise them into a displayable range first).

They are tiny, so do not expect much detail — but you should be able to see that some have become edge detectors and others colour-opponent patches. Nobody wrote those; they came out of gradient descent.

Compare them to the kernels we wrote by hand in the CV sessions.

8. What to submit

One file — a notebook or a `.py`.

1. **Your shape and parameter table**, filled in by hand before coding (a photo is fine), plus the model summary and parameter count that confirm it.
2. **Both models**, trained with identical settings.

3. **The comparison table** — parameters, test accuracy, time per epoch, for both.
4. **Loss curves** for both models on one plot.
5. **The first-layer filter visualisation.**
6. **Written answers**, two to four sentences each:
   - What does the CNN assume about images that the MLP does not? Name both assumptions and say what each one saves you.
   - An MLP sees the image as 3072 independent numbers. What happens to its learned weights if every image is shifted two pixels to the right? What happens to the CNN's?
7. **Two to four sentences** on the first real bug you hit and how you found it. Be specific.